Universidade Vigo

Practica I Tecnologias de Registro Distribuido

Francisco Javier Pereira Alonso, Javier Alejandro Amorosi Gomez

27 de octubre de 2026

1 Ejercicio 5

1.1 Analisis y Definicion del Escenerio

Se eligio como caso de uso para la implementacion de un Smart Contract utilizando el lenguaje de programacion *Solidity* el siguiente escenario;

- Generacion y gestion de pagos futuros.

Particularmente, el Smart Contract que se quiere implementar debe soportar la operacion y la gestion de pagos entre particulares que se determina deben ser recibidos por un beneficiario en un momento en el futuro. La necesidad que se tiene en muchas ocaciones de cumplir con un pago para el cual ambas partes estan de acuerdo, como puede ser el pago de un servicio a 7,15 o 30 dias o el pago de un contrato a fin de mes que ya fue pactado por ambas partes, o incluso el pago de una subscripcion por el uso de un servicio fue la motivacion principal para el desarrollo de esta actividad. La existencia de NFTs, particularmente el estandar ERC721 de Ethereum, se presenta como una forma de demostrar por parte del *deudor* del pago al *beneficiario* del mismo que el pago va a ocurrir en un momento dado. Al mismo tiempo, la posibilidad de penalizar a un *deudor* con cuentas pendientes en caso de no cumplir con una deuda mediante la implementacion de la logica de los pagos en un Smart Contract refuerza la motivacion para la implementacion de este sistema con la ayuda de tecnologias de registro distribuidos como puede ser la red Ethereum.

1.2 Diseño

1.2.1 Usuarios del Sistema

En el escenario descrito anteriormente, se pueden identificar dos tipos de usuarios diferentes:

1. Deudores, o aquellos usuarios que deben cumplir con un pago
2. Acreedores, o aquellos usuarios que son beneficiarios de un pago que un deudor debe cumplir

1.2.2 Estructura del Contrato

El contrato gira en torno de un pago, que es una deuda a saldar antes de un momento determinado. Un Pago se compone entonces de;

- Monto total a pagar
- Monto total pagado
- Acreedor del pago
- Deudor del pago
- Plazo de vencimiento del pago

Dicho esto, dado un Pago en particular se pueden realizar las siguientes operaciones basicas

- Pagar un monto
- Cobrar el pago

Ademas, algunas invariantes que deben cumplir los pagos son;

1. El monto pactado es inalterable
2. El momento de cobro es inalterable
3. Un Deudor no puede transferir el pago

Finalmente, algunas operaciones que se pueden realizar utilizando las condiciones anteriormente mencionadas mas alla de las operaciones basicas son;

1. Un Acreedor que tambien es Deudor puede pagar la deuda con un Pago cuando
   (a) El monto que debe es menor o igual al monto que se le debe
   (b) El plazo de vencimiento del monto debido es posterior o igual al plazo de vencimiento del Pago del cual es acreedor.
2. Un Acreedor puede transferir un pago a alguien mas

1.2.3 Casos de Uso

Se busca cubrir las siguientes situaciones con la implementacion del Smart Contract;

1. Un Acreedor puede reconocer al deudor de un pago que se le debe
2. Un Deudor debe pagar sus pagos vencidos antes de poder generar nuevos Pagos
3. Un Acreedor puede saldar montos utilizando pagos que le son debidos

2 Implementacion

Se implemento el Smart Contrar para el cual se proporciona el codigo fuente en el repositorio correspondiente.

El contrato extiende la funcionalidad del estandar ERC721 utilizando la libreria de Smart Contracts de Openzepellin e implementa las siguientes operaciones;

1. createPayment
   (a) Genera un nuevo pago del cual la persona que lo crea es deudor
2. pay
   (a) Salda parte o toda la deuda de un pago
3. claim
   (a) Cobra un pago
4. slashPayer
   (a) Marca a un deudor como moroso, impidiendole generar nuevos pagos
5. payWithPayment
   (a) Salda una deuda utilizando un pago del cual el deudor es acreedor, esto ocurre generando un nuevo Pago partiendo del original.
   (b) Si el monto a saldar es menor al monto adeudado del pago original, el pago original ve disminuido su monto.
   (c) Otra forma de ver esta funcion, es como la transferencia de obligaciones de un deudor a otro.

Ademas, se modifica el comportamiento de la transferencia del estandar original (ERC721) para que;

- La transferencia de Pagos solamente pueda ser realizada por el acreedor, modificando unicamente el acreedor original.

2.1 Optimizacion de Gas

Se utilizaron las siguientes tecnicas de optimizacion de gas

- Payment.dueTime es un uint48, segun se recomienda en el documento correspondiente
- Lectura de variables de storage una unica vez cuando la variable se lee entera
  - Esto es, se evita cargar un Pago completo en memoria si se puede evitar.
- Utilizacion de punteros a storage cuando solamente hay que leer parte de un Pago
  - Esto es, cuando checkeamos valores diferentes al vencimiento (que es lo ultimo de un pago)
- Escritura de variables de storage una unica vez.
- Se evito negar condiciones.
- Se utilizaron namedReturns
- Se utilizaron errores en vez de require

3 Pruebas

Se realizaron pruebas del contrato utilizando RemixIDE para probar la funcionalidad en los siguientes escenarios;

Una primera prueba busco dar cuenta de que la implementacion permitia a un acreedor cobrar su deuda saldada. Un este segundo caso, se busca mostrar que un deudor con una deuda no saldada es incapaz de generar nuevos pagos. Finalmente un caso en el que se busco probar la funcionalidad de pago de deuda utilizando lo debido por un tercer acreedor.

Para estos tres escenarios diferentes, se deja un diagrama para poder ver las interacciones de multiples cuentas con el contrato.

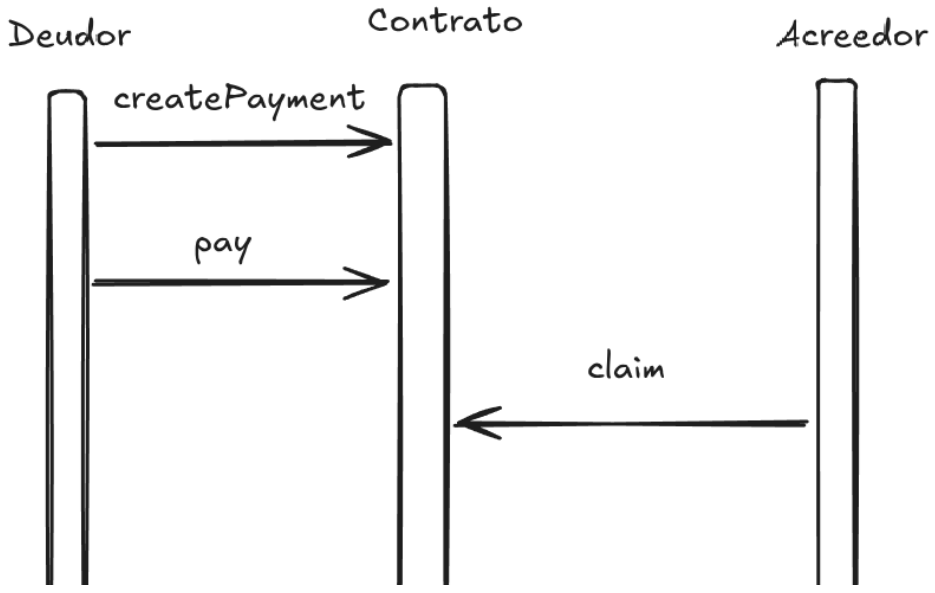


Figure 1: Creacion de deuda y pago en plazo

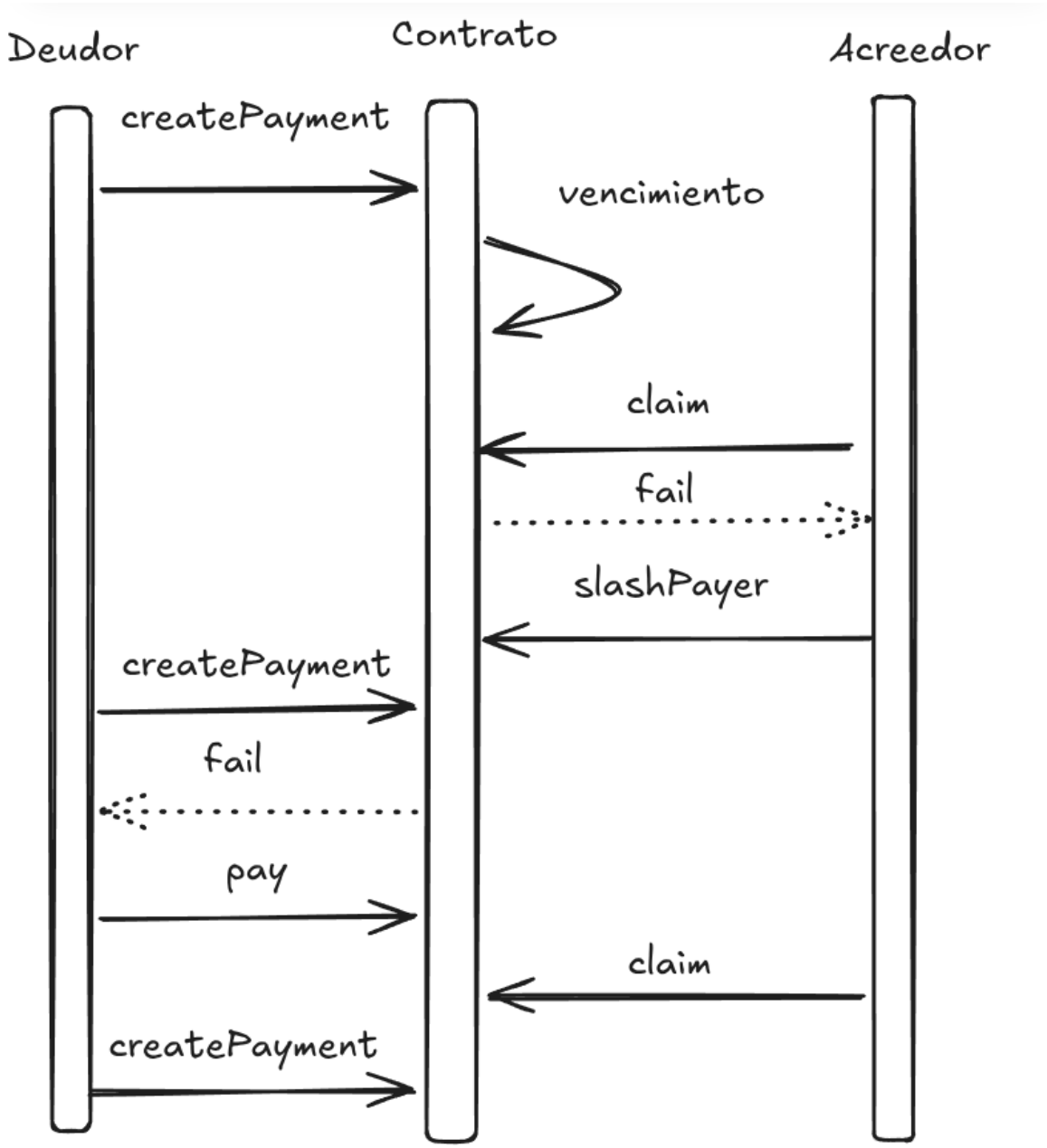


Figure 2: Imposibilidad de crear nuevos pagos

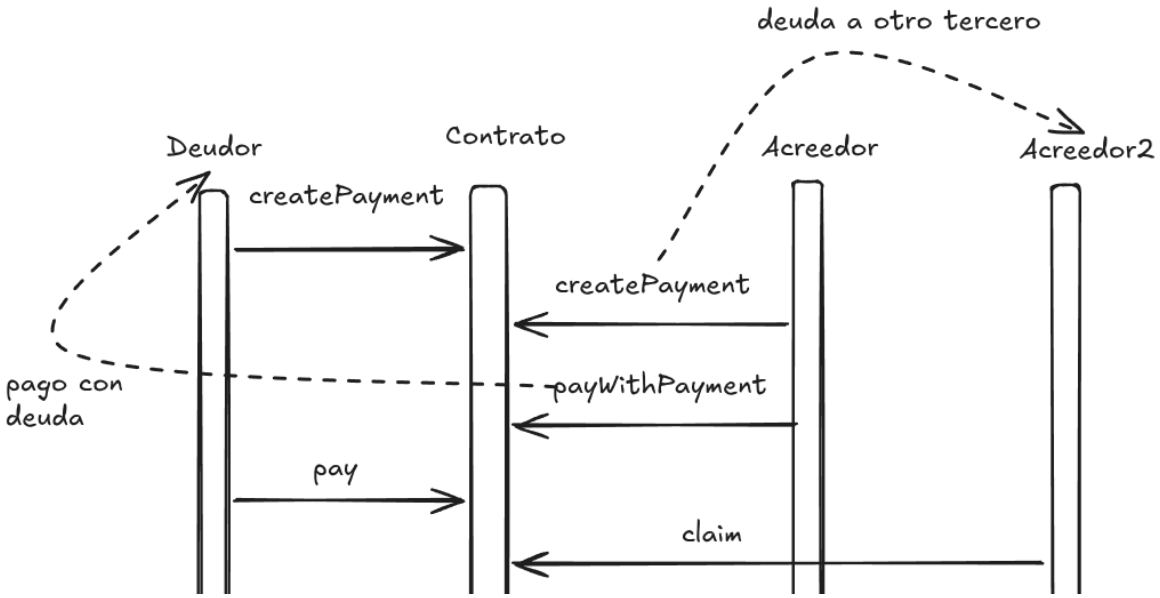


Figure 3: Pago de deuda con deuda